German Jordanian University

School of Electrical Engineering and Information Technology

Department of Computer Engineering

Evaluation of Wireless Mesh Network Routing Protocols: AODV vs. DSDV Using OMNeT++/INET Simulation

By

Abdullah Mohammad Khair Ababneh

Supervised by

Dr. Ala'a Khalifeh

Associate Professor, Department of Computer Engineering

German Jordanian University, Amman, Jordan

A Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Engineering

4 March 2026

Dedication

To my family — my greatest source of strength and inspiration.

With deep gratitude, I dedicate this work to my incredible family. Your unyielding support and the sacrifices you made to help me pursue my education mean more than words can express. Your belief in my potential and your steadfast encouragement have been the bedrock of my success.

This achievement is not just a personal victory but a tribute to our collective journey. Your love has been the wind beneath my wings, propelling me forward even in the face of challenges. To you, my pillars of strength, I owe this success as much as to myself.

Abdullah Mohammad Khair Ababneh

Acknowledgments

First and foremost, I express my deepest gratitude to Allah the Almighty, the Most Gracious, and the Most Merciful, for His continuous blessings throughout my academic journey and for granting me the strength and clarity of mind to complete this thesis. May His blessings extend to His final Prophet Muhammad (peace be upon him), his family, and his companions.

A special and heartfelt acknowledgment goes to my supervisor, Dr. Ala'a Khalifeh, Associate Professor at the German Jordanian University. Dr. Khalifeh, your invaluable guidance, patience, and constructive feedback throughout every stage of this project have been indispensable. Thank you for your mentorship, for always making time to answer my questions, and for inspiring me with your deep knowledge of wireless networking systems.

I extend my gratitude to all the faculty members of the Department of Computer Engineering at GJU whose teaching laid the technical foundation that made this research possible, particularly in subjects such as Computer Networks, Advanced Networking Technologies, and Embedded Systems.

I also wish to acknowledge the developers and maintainers of the OMNeT++ simulation environment and the INET Framework, whose open-source tools made this research feasible without requiring access to expensive physical hardware.

To my beloved family — thank you for your unwavering support, your patience during the long hours I spent at my desk, and your trust in my abilities. This achievement belongs to all of us.

To my colleagues and friends who offered encouragement, shared ideas, and helped me debug configurations at 2 am — you know who you are, and I am grateful.

Abdullah Mohammad Khair Ababneh
Amman, Jordan, 4 March 2026

Abstract

Wireless Mesh Networks (WMNs) have emerged as a key enabling technology for extending wireless connectivity in dynamic, infrastructure-limited, and resource-constrained environments, with applications spanning smart cities, industrial IoT, community broadband, and emergency communications. The routing protocol chosen for a WMN directly governs the network's performance in terms of throughput, end-to-end delay, energy consumption, and delivery reliability. Despite the large body of theoretical literature on routing protocol trade-offs, quantitative, simulation-based evidence comparing these protocols under controlled, repeatable conditions remains valuable for guiding protocol selection in practice.

This thesis presents a systematic simulation-based evaluation of two widely-studied WMN routing protocols: the reactive Ad hoc On-Demand Distance Vector (AODV) protocol (RFC 3561) and the proactive Destination-Sequenced Distance-Vector (DSDV) protocol. Simulations were conducted using OMNeT++ 6.3 with the INET 4.5 framework, modelling a stationary wireless mesh network in a $1000\,\mathrm{m} \times 1000\,\mathrm{m}$ area with IEEE 802.11g radio links operating at 54 Mbps. Five network densities were tested ($N \in \{10, 20, 30, 40, 50\}$ nodes), and a many-to-one UDP traffic pattern was used to simulate IoT data collection scenarios. Five performance metrics were recorded: total bytes received (throughput), packet reception count, end-to-end packet delay, residual energy capacity, and packet drop rate due to missing routes.

Results reveal a density-dependent performance crossover. In sparse networks ($N = 10$), DSDV exhibits lower median delay because routing tables are already populated at simulation start, whereas AODV incurs significant route-discovery latency. However, as network density increases beyond $N = 20$, AODV consistently outperforms DSDV across all metrics. At $N = 50$, AODV delivers approximately 35% higher throughput (2.5 MB vs. 1.85 MB) and maintains zero packet drops due to routing failures, while DSDV suffers from 121 or more queue-overflow drops caused by the increased control traffic flooding the IEEE 802.11 MAC layer. AODV also consumes approximately 8% less energy at $N = 50$ due to the elimination of periodic route broadcasts.

The MAC-layer contention mechanism responsible for DSDV's collapse at high densities

is demonstrated through contention window (CW) escalation traces showing exponential back-off growth from CW = 31 to CW = 511 slots and retry counter abort at the 7-retry limit. This provides physical-layer evidence that DSDV's performance degradation is not algorithmic but driven by channel saturation. Deployment guidelines derived from these findings recommend AODV for large-scale ($N > 20$), reliability-critical IoT WMNs, and DSDV for small ($N < 20$), low-density, latency-sensitive applications.

Keywords: Wireless Mesh Networks, AODV, DSDV, OMNeT++, INET, routing protocol evaluation, IoT, MAC contention, binary exponential back-off, energy consumption.

Contents

Authorization Form i

Declaration of Originality ii

Dedication iii

Acknowledgments iv

Abstract v

List of Abbreviations xiii

1 Introduction 1

1.1 Background and Motivation 1
1.2 Problem Statement 2
1.3 Research Objectives 3
1.4 Thesis Contributions 3
1.5 Thesis Structure 4

2 Wireless Mesh Networks: Background and Architecture 5

2.1 Definition and Characteristics 5
2.2 Network Architecture 6
2.2.1 Infrastructure Mesh 6
2.2.2 Client Mesh 6
2.2.3 Hybrid Mesh 6
2.3 Key Performance Metrics 6
2.3.1 Throughput 7
2.3.2 End-to-End Delay 7
2.3.3 Packet Delivery Ratio (PDR) 7
2.3.4 Energy Consumption 7
2.3.5 Routing Overhead 7
2.4 Applications of Wireless Mesh Networks 7

2.5 Challenges in WMN Routing 8

3 Routing Protocols and MAC Layer Theory 10
3.1 Classification of Routing Protocols 10
3.1.1 Proactive (Table-Driven) Protocols 10
3.1.2 Reactive (On-Demand) Protocols 11
3.1.3 Hybrid Protocols 11
3.2 DSDV: Destination-Sequenced Distance Vector 12
3.2.1 Overview and History 12
3.2.2 Routing Table Structure 12
3.2.3 Update Mechanism 13
3.2.4 Convergence and Stale Routes 13
3.2.5 DSDV Algorithm Summary 14
3.3 AODV: Ad hoc On-Demand Distance Vector 14
3.3.1 Overview and Standardisation 14
3.3.2 Route Discovery 14
3.3.3 Route Maintenance 15
3.3.4 Sequence Number Management 15
3.3.5 AODV Algorithm Summary 15
3.4 IEEE 802.11 DCF and MAC Layer Contention 15
3.4.1 Distributed Coordination Function (DCF) 15
3.4.2 Binary Exponential Back-off 16
3.4.3 Retry Limit and Frame Dropping 17
3.5 Related Work 17

4 Simulation Methodology 19
4.1 Simulation Platform 19
4.2 Network Model 20
4.2.1 Network Description (NED) File 20
4.2.2 Node Count Parameterisation 21
4.3 Node Placement and Spatial Density 22
4.4 Mobility Model 23
4.5 Radio and MAC Configuration 23
4.6 Energy Model 24
4.7 Traffic Model 24
4.8 Performance Metrics and Recording 24
4.9 Simulation Configurations 25

5 Results and Discussion 27
5.1 Overview of Results 27

5.2 End-to-End Delay . . . 27
5.2.1 AODV Delay: U-Shaped Curve . . . 27
5.2.2 DSDV Delay: Inverse U-Shaped Curve . . . 28
5.2.3 Crossover Analysis . . . 29
5.3 Throughput and Packet Delivery . . . 29
5.3.1 Total Bytes Received . . . 29
5.3.2 Packet Reception Count . . . 30
5.4 Packet Drop Analysis . . . 30
5.4.1 AODV: Zero Route-Miss Drops . . . 30
5.4.2 DSDV: Drops at Sparse and Dense Extremes . . . 31
5.5 MAC-Layer Contention Evidence . . . 31
5.5.1 Contention Window Escalation . . . 32
5.5.2 Why DSDV Saturates First . . . 32
5.6 Energy Consumption . . . 33
5.6.1 Residual Energy at `host[25]` . . . 33
5.6.2 AODV Energy Advantage at High Density . . . 33
5.6.3 DSDV Energy Advantage at Low Density . . . 34
5.7 Summary of Results . . . 35
5.8 Trade-off Analysis and Deployment Implications . . . 35
5.8.1 The Scalability Dilemma . . . 35
5.8.2 The Discovery Latency Penalty . . . 35
5.8.3 The MAC-Layer Coupling . . . 36

6 Conclusion and Future Work 37
6.1 Conclusion . . . 37
6.2 Deployment Guidelines . . . 38
6.2.1 When to Choose DSDV . . . 38
6.2.2 When to Choose AODV . . . 38
6.2.3 Decision Matrix . . . 39
6.3 Future Work . . . 39
6.4 Final Remarks . . . 40

A Simulation Configuration Files 43
A.1 Complete `omnetpp.ini` . . . 43
A.2 Complete `MeshNetwork.ned` . . . 45

B Mathematical Background 47
B.1 Bianchi's IEEE 802.11 Saturation Throughput Model . . . 47
B.2 Retry-Limit Frame Drop Probability . . . 48

List of Figures

4.1 OMNeT++ IDE NED Design view of the `MeshNetwork` model. The submodule palette on the right lists available router types, including `AodvRouter`, `DsdvRouter`, and `ManetRouter`. The canvas shows the three submodules: `configurator`, `radioMedium`, and the parameterised `host[numNodes]` array. 20
4.2 NED source code for `MeshNetwork` shown in the OMNeT++ IDE Source tab. The network imports `Ipv4NetworkConfigurator`, `ManetRouter`, and `RadioMedium`. Node placement and radio-medium parameters are set in `omnetpp.ini` at runtime. 21
4.3 OMNeT++ Analysis Tool showing all 10 simulation runs (`AODV_Test` and `DSDV_Test`, each at $N \in \{10, 20, 30, 40, 50\}$). The result-name filter panel on the right lists ARP and packet-level statistics available for analysis. 22

5.1 End-to-end packet delay (packet lifetime) distribution for AODV and DSDV at $N \in \{10, 20, 30, 40, 50\}$. Box plots show median (centre line), interquartile range (box), and 1.5×IQR whiskers. AODV shows a U-shaped delay profile; DSDV shows an inverse trend. 28
5.2 Total bytes received at the sink (`host[0]`) for AODV and DSDV across all five network densities. 29
5.3 Total packets received at the sink for AODV and DSDV. 30
5.4 Packet drops due to "no route found" errors for AODV and DSDV. . . 31
5.5 OMNeT++ Qtenv simulation view during the AODV run at $N = 30$, showing the mesh topology with simultaneous ARP reply signals (red arrows) radiating from the sink node. The event log confirms rapid radio state transitions from `IDLE` to `RECEIVING`, illustrating the high channel activity that drives energy consumption at moderate-to-high density. . 33

5.6 Residual energy capacity at `host[25]` over simulation time for $N = 50$, comparing AODV (blue) and DSDV (orange). The steeper slope for DSDV reflects higher duty-cycle TX/RX activity caused by periodic routing broadcasts, resulting in approximately 8% higher total energy consumption. 34

List of Tables

3.1 Comparison of routing protocol classes for WMNs. 12

4.1 Expected number of neighbours per node as a function of network size. 23
4.2 Complete simulation parameter reference. 26

5.1 Contention-window escalation after successive collisions (802.11g). . . . 32
5.2 Summary of simulation results: AODV vs. DSDV across all metrics and network sizes. A ✓ indicates the superior protocol for each metric and density. 35

6.1 Protocol selection decision matrix for WMN deployments. 39

B.1 Frame-drop probability vs. collision probability ($m_r = 7$ retries). 48

List of Abbreviations

ACK	Acknowledgment
AODV	Ad hoc On-Demand Distance Vector
AP	Access Point
ARP	Address Resolution Protocol
BEB	Binary Exponential Back-off
BSS	Basic Service Set
CBR	Constant Bit Rate
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
CW	Contention Window
DCF	Distributed Coordination Function
DIFS	DCF Inter-Frame Space
DSDV	Destination-Sequenced Distance Vector
DYMO	Dynamic MANET On-demand
EIFS	Extended Inter-Frame Space
HWMP	Hybrid Wireless Mesh Protocol
IEEE	Institute of Electrical and Electronics Engineers
INET	INET Framework for OMNeT++
IoT	Internet of Things
IP	Internet Protocol
IQR	Interquartile Range
LAN	Local Area Network

MAC Medium Access Control

MANET Mobile Ad hoc Network

MBR Maximum Bit Rate

MPR Multipoint Relay

MTU Maximum Transmission Unit

NED Network Description (OMNeT++ language)

OLSR Optimized Link State Routing Protocol

OMNeT++ Objective Modular Network Testbed in C++

QoS Quality of Service

RERR Route Error (AODV control message)

RF Radio Frequency

RREP Route Reply (AODV control message)

RREQ Route Request (AODV control message)

RTT Round-Trip Time

RWP Random Waypoint (mobility model)

SIFS Short Inter-Frame Space

SNR Signal-to-Noise Ratio

TCP Transmission Control Protocol

TTL Time to Live

UDP User Datagram Protocol

WMN Wireless Mesh Network

WLAN Wireless Local Area Network

ZRP Zone Routing Protocol

Chapter 1

Introduction

1.1 Background and Motivation

The proliferation of connected devices in the Internet of Things (IoT) era has fundamentally changed the expectations placed on wireless networking infrastructure. Traditional access-point-centric wireless networks, where all communication is mediated through a fixed access point, struggle to scale gracefully to large numbers of devices distributed over wide areas. They create single points of failure, require expensive cabling infrastructure, and cannot self-organise or self-heal in the event of component failure.

Wireless Mesh Networks (WMNs) address these limitations by distributing the routing and forwarding functions across all participating nodes. In a WMN, every device acts simultaneously as a data source, a relay, and — when connected to the wired backbone — a gateway. Packets traverse the network over multiple wireless hops, following paths computed by a dedicated routing protocol. This multi-hop architecture provides geographic coverage far beyond the reach of any single radio transmitter, allows the network to re-route around failed or congested links, and can be deployed rapidly without pre-existing infrastructure.

The routing protocol is, consequently, the central design choice in any WMN deployment. It determines how routes are discovered and maintained, how much overhead is generated in control traffic, how quickly the network responds to topological changes, and how energy is consumed at each node. The two broad families of routing protocols that have been most widely studied are:

- **Proactive (table-driven) protocols**, which continuously maintain routing tables for all destinations, trading high control overhead for immediately-available routes.

- **Reactive (on-demand) protocols**, which discover routes only when a communication session requires one, trading an initial discovery delay for lower steady-state control overhead.

The Destination-Sequenced Distance-Vector (DSDV) protocol [1] is the canonical proactive protocol, extending the classic Bellman-Ford algorithm with sequence numbers to eliminate routing loops. The Ad hoc On-Demand Distance Vector (AODV) protocol [2], standardised as RFC 3561, is the canonical reactive protocol and has been widely deployed in both research and production MANET and WMN systems.

Despite extensive theoretical and simulation-based comparisons in the literature, many studies are conducted under idealised channel models, simplified traffic patterns, or without rigorous control of the MAC-layer parameters that determine how control and data traffic compete for the shared wireless medium. This gap motivates the present work: a carefully controlled simulation study using a high-fidelity IEEE 802.11 MAC and physical-layer model, a realistic many-to-one traffic pattern representative of IoT data collection, and a systematic sweep over network density that reveals the crossover behaviour between the two protocols.

1.2 Problem Statement

Selecting a routing protocol for a WMN deployment involves navigating a fundamental tension between two competing design objectives:

1. **Route availability.** A node must have a valid route to the destination before it can transmit data. In sparse networks, routes may not yet be available (reactive protocols) or may have gone stale (proactive protocols with convergence lag).
2. **Channel efficiency.** The wireless medium is shared. Every byte of routing control traffic consumes airtime that could otherwise carry application data. In dense networks, proactive protocols that broadcast routing tables periodically can saturate the medium, leading to MAC-layer congestion and cascading packet loss.

The precise point at which the dominant bottleneck shifts from route availability to channel efficiency — the *crossover density* — is a function of the specific protocol parameters, radio technology, traffic load, and network size. This crossover is poorly characterised in the existing literature for the specific combination of AODV, DSDV, IEEE 802.11g, and many-to-one IoT traffic patterns.

Furthermore, most existing studies report only application-layer performance metrics (throughput, delay, packet delivery ratio) without tracing the physical mechanism responsible for observed performance degradation at the MAC layer. This makes it

difficult for practitioners to reason about whether a performance problem is caused by sub-optimal routing logic or by low-level channel saturation — an important distinction when diagnosing and tuning deployed systems.

1.3 Research Objectives

This thesis pursues four specific research objectives:

1. **Quantify the crossover density.** Determine the value of N at which AODV begins to outperform DSDV across the five performance metrics studied: throughput, packet count, end-to-end delay, energy consumption, and packet drop rate.
2. **Explain the crossover mechanism.** Trace the causal chain from routing protocol behaviour, through MAC-layer contention, to observed application-layer performance, providing quantitative evidence at each level.
3. **Provide MAC-layer evidence.** Extract and analyse contention-window escalation statistics from the OMNeT++/INET simulation to demonstrate the physical-layer mechanism responsible for DSDV's performance collapse at high densities.
4. **Derive deployment guidelines.** Translate the quantitative findings into actionable engineering guidance for practitioners selecting between AODV and DSDV for WMN deployments in IoT and other application domains.

1.4 Thesis Contributions

The principal contributions of this thesis are:

- A fully reproducible OMNeT++ 6.3 / INET 4.5 simulation setup for comparing AODV and DSDV in a WMN context, with all configuration files provided in Appendix A.
- Empirical demonstration of the AODV/DSDV performance crossover at approximately $N = 20$ nodes, with quantitative data for throughput, delay, energy, and drop rate across all five tested densities.
- MAC-layer evidence linking DSDV's performance degradation at $N = 50$ to contention-window escalation and the 7-retry abort limit defined by the IEEE 802.11 standard.
- A comprehensive energy analysis showing AODV's 8% energy advantage at high densities and explaining the reversal at $N = 10$.

- Protocol-selection guidelines for WMN practitioners, structured as a decision matrix covering network size, latency requirements, reliability requirements, and energy constraints.

1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 provides background on WMN architecture, characteristics, and applications. Chapter 3 reviews the theoretical foundations of proactive and reactive routing, describes the AODV and DSDV protocols in detail, and analyses the IEEE 802.11 DCF MAC layer. Chapter 4 describes the simulation methodology, including the network model, parameter choices, and metric collection procedures. Chapter 5 presents and analyses the simulation results, with particular attention to the trade-offs between the two protocols. Chapter 6 summarises the findings, provides deployment guidelines, and outlines directions for future work.

Chapter 2

Wireless Mesh Networks: Background and Architecture

2.1 Definition and Characteristics

A Wireless Mesh Network (WMN) is a communication network built from radio nodes organised in a mesh topology, where every node is capable of relaying data on behalf of its neighbours [3]. Unlike traditional single-hop wireless networks in which every client communicates through a central access point, WMN nodes forward traffic for one another over multiple wireless hops, making the network self-organising and self-healing.

The distinguishing characteristics of WMNs are:

Multi-hop communication. Packets may traverse several intermediate relay nodes to reach their destination, extending the effective coverage area far beyond the range of any single radio transmitter.

Self-organisation. Nodes discover their neighbours and compute routes automatically, without manual configuration. New nodes joining the network are incorporated with minimal disruption.

Self-healing. When a link or node fails, the network re-routes traffic around the failure. This property is provided by the routing protocol and is one of the primary motivations for using WMNs in critical-infrastructure applications.

Partial infrastructure independence. A WMN can operate without any pre-existing wired infrastructure, though in practice one or more gateway nodes connect the mesh to the Internet or a backbone network.

Scalability tension. While multi-hop forwarding extends coverage, larger networks require more complex routing, more control traffic, and more relay hops that

add delay and consume energy. Managing this tension is the central challenge in WMN design.

2.2 Network Architecture

WMNs are typically classified into three architectural categories [3, 4]:

2.2.1 Infrastructure Mesh

In an infrastructure WMN, a set of dedicated mesh routers forms a backbone. These routers are typically mains-powered, have directional antennas, and run advanced multi-radio, multi-channel configurations to maximise backbone throughput. Client devices (laptops, smartphones, sensors) connect to the backbone through the nearest mesh router. This architecture is used in large-scale community broadband networks and enterprise campus networks.

2.2.2 Client Mesh

In a client WMN, the client devices themselves participate in routing. There are no dedicated routers; every node both generates data and forwards traffic for its peers. This architecture suits ad hoc deployments such as disaster-relief communications and vehicular networks (VANETs). The main challenges are device heterogeneity and energy constraints, since the routing burden falls on battery-powered client devices.

2.2.3 Hybrid Mesh

A hybrid WMN combines dedicated mesh routers with mesh-capable client devices. Clients can communicate directly with each other and with nearby mesh routers, while the routers provide backbone connectivity and Internet access. This is the most common commercial deployment model and the architecture implicitly modelled in this thesis, where all nodes are `ManetRouter` instances capable of both application traffic and routing.

2.3 Key Performance Metrics

Evaluating a WMN routing protocol requires measuring multiple, sometimes conflicting, performance dimensions:

2.3.1 Throughput

Throughput is the rate at which useful application data is delivered to the destination, typically measured in bytes per second or as a total byte count over the simulation period. High routing overhead reduces throughput by consuming channel capacity with control messages.

2.3.2 End-to-End Delay

End-to-end delay is the time elapsed between a packet being generated at the source and being received at the destination. It includes: queuing delay (waiting for the MAC layer), transmission delay (radio transmission time), propagation delay (signal travel time), and routing delay (route discovery time for reactive protocols). In IoT applications, delay requirements vary widely: industrial control requires sub-millisecond response times, while environmental monitoring can tolerate delays of several seconds.

2.3.3 Packet Delivery Ratio (PDR)

The PDR is the fraction of transmitted packets that are successfully received at the destination. Drops occur due to missing routes (for reactive protocols during route discovery, or for proactive protocols during convergence after a topology change), buffer overflow, or MAC-layer transmission failure after exhausting all retransmission attempts.

2.3.4 Energy Consumption

Energy consumption is a critical metric in IoT applications where nodes are battery-powered. Routing protocols that generate high volumes of control traffic cause radios to spend more time in the transmit and receive states, which are the most energy-intensive radio states.

2.3.5 Routing Overhead

Routing overhead is the ratio of control bytes to total bytes transmitted. A high routing overhead indicates that a large fraction of channel capacity is consumed by protocol maintenance rather than application data.

2.4 Applications of Wireless Mesh Networks

WMNs have been deployed across a wide range of application domains:

Community and municipal broadband. Cities in Europe, Asia, and North America have deployed WMNs to provide low-cost Internet access in public spaces and underserved areas. Notable examples include the Madrid city-wide WMN and municipal deployments in rural Bangladesh.

Industrial IoT and smart manufacturing. Factories use WMNs to connect sensors, actuators, and programmable logic controllers (PLCs) without the cost and rigidity of wired Ethernet. The self-healing property is particularly valuable in environments where equipment is rearranged frequently.

Smart grid and utility monitoring. Electric utilities deploy WMNs to connect smart meters, substation monitoring equipment, and demand-response control systems. Many-to-one traffic patterns — all meters reporting to a central collector — are characteristic of this use case, and are the pattern modelled in this thesis.

Disaster relief and emergency communications. When earthquake or flood damage destroys fixed infrastructure, first responders can deploy portable WMN nodes in minutes to restore communications. The infrastructure-free nature of client-mesh architectures is essential in these scenarios.

Vehicular Networks (VANETs). Vehicles equipped with IEEE 802.11p (DSRC) or cellular-V2X radios form mobile mesh networks to exchange safety messages, traffic information, and infotainment content. VANETs present extreme mobility challenges for routing protocols.

Agricultural and environmental monitoring. WMNs connect soil moisture sensors, weather stations, and irrigation controllers across large fields, enabling precision agriculture without expensive cabling.

2.5 Challenges in WMN Routing

Routing in WMNs is significantly more challenging than routing in wired networks for several reasons:

Shared wireless medium. All nodes within radio range compete for the same channel. The IEEE 802.11 CSMA/CA protocol manages this contention, but at the cost of latency and reduced throughput as the number of competing nodes increases.

Hidden terminal problem. Node A can hear node B, and node C can hear node B, but A cannot hear C. If A and C transmit simultaneously, they collide at B. The IEEE 802.11 RTS/CTS mechanism mitigates but does not eliminate this problem.

Many-to-one funnelling. In IoT data-collection scenarios, all nodes transmit to a single gateway or sink. Traffic density increases near the sink (the hot-spot effect), creating severe MAC-layer congestion in the nodes closest to the sink that must relay traffic from many sources.

Routing loop and stale-route problems. Distance-vector protocols are susceptible to routing loops and can maintain stale routes that point to nodes that have moved or become unreachable. DSDV's sequence numbers partially address this, but convergence is not instantaneous.

Energy heterogeneity. In mixed deployments, some nodes are mains-powered and others are battery-powered. A protocol that maximises network throughput may exhaust battery-powered nodes first, partitioning the network.

Chapter 3

Routing Protocols and MAC Layer Theory

3.1 Classification of Routing Protocols

Routing protocols for ad hoc and mesh networks are broadly classified into three categories based on when and how they compute routes [5]:

3.1.1 Proactive (Table-Driven) Protocols

Proactive protocols maintain a continuously updated routing table that contains entries for all reachable destinations in the network. Whenever a topology change is detected — a link failure, a new node joining, or a metric update — the protocol propagates update messages throughout the network to synchronise all routing tables.

Advantages:

- Routes are immediately available when a transmission session begins; no discovery delay.
- Predictable and bounded latency, since the routing layer never needs to wait for route discovery.
- Simple implementation: standard distance-vector or link-state algorithms adapted for wireless media.

Disadvantages:

- High control overhead, especially in dense networks where many nodes exchange routing updates periodically.

- Routing table entries for destinations that are never communicated with consume memory and channel capacity unnecessarily.
- Convergence delay after topology changes can leave stale routes in the table, potentially causing black-hole or routing-loop problems.

Examples of proactive protocols include DSDV [1], OLSR [6], and BATMAN.

3.1.2 Reactive (On-Demand) Protocols

Reactive protocols do not maintain routing tables for all destinations. Instead, they initiate a route discovery procedure — typically by flooding a Route Request (RREQ) message throughout the network — only when a node needs to communicate with a destination for which it has no valid route. A Route Reply (RREP) is sent back from the destination to the source along the reverse path, establishing the route. The route is cached and used for subsequent packets to the same destination until it expires or a link failure is detected.

Advantages:

- Low steady-state control overhead; no periodic broadcasts when the network is idle.
- Scales to larger networks because routing state is built on demand.
- Energy-efficient: radios spend more time in the receive or sleep state rather than transmitting periodic updates.

Disadvantages:

- Route discovery latency on first contact with a destination, which can be significant in sparse networks where the RREQ must traverse many hops.
- RREQ flooding can cause a broadcast storm in dense networks if not carefully controlled.
- Maintaining routes for long-lived sessions requires periodic HELLO messages, adding some overhead.

Examples of reactive protocols include AODV [2], DSR [7], and DYMO.

3.1.3 Hybrid Protocols

Hybrid protocols combine proactive and reactive mechanisms. They maintain routing tables proactively within a local zone of a configurable radius, and use reactive flooding for destinations outside the zone. The Zone Routing Protocol (ZRP) and HWMP (the

protocol mandated by IEEE 802.11s for standardised mesh networking) are prominent examples.

Hybrid protocols aim to capture the low-latency benefit of proactive routing for nearby nodes while limiting overhead by restricting proactive maintenance to the local neighbourhood. However, the zone radius must be tuned carefully to achieve the desired trade-off, introducing an additional configuration parameter.

Table 3.1 summarises the key characteristics of the three protocol classes.

Table 3.1: Comparison of routing protocol classes for WMNs.

Property	**Proactive**	**Reactive**	**Hybrid**
Route availability	Always available	Discovery delay	Zone: immediate; outside:
Control overhead	High (periodic)	Low (on-demand)	Medium
Memory usage	High (full table)	Low (active only)	Medium
Scalability	Poor in dense	Better	Good with tuning
Delay predictability	High	Low (first packet)	Medium
Energy efficiency	Lower	Higher	Medium
Best for	Small, latency-critical	Large, infrequent traffic	Large, mixed traffic

3.2 DSDV: Destination-Sequenced Distance Vector

3.2.1 Overview and History

DSDV was proposed by Perkins and Bhagwat in 1994 [1] as an adaptation of the classic Bellman-Ford distance-vector algorithm for mobile ad hoc networks. The key innovation was the introduction of sequence numbers associated with each route entry: every destination maintains a monotonically increasing sequence number, and route updates bearing higher sequence numbers supersede older entries. This prevents the *counting to infinity* problem that plagues standard Bellman-Ford in dynamic topologies.

3.2.2 Routing Table Structure

Every DSDV node maintains a routing table with one entry per known destination. Each entry contains:

- **Destination address:** the IP or MAC address of the destination node.
- **Next hop:** the address of the neighbouring node to which packets should be forwarded.
- **Metric:** typically the hop count to the destination, though other metrics (delay, ETX) can be used.

- **Sequence number:** an even number assigned by the destination when it initiates an update, or an odd number assigned by an intermediate node to mark a broken route. Higher is more recent; odd numbers signal link failure.
- **Install time:** the time at which the entry was last updated, used to expire stale routes.

3.2.3 Update Mechanism

DSDV uses two types of update messages:

Full dump. A complete routing table broadcast, sent periodically (typically every 30–60 seconds) or when the routing table changes substantially. Full dumps allow newly joining nodes to acquire the complete network topology quickly.

Incremental update. A partial table broadcast containing only the entries that have changed since the last full dump. Incremental updates are sent more frequently (typically every 15 seconds) to propagate link-state changes rapidly. Their smaller size reduces channel load between full dumps.

Upon receiving an update, a node accepts a new entry if:

1. The incoming sequence number is greater than the stored one (newer information), regardless of metric; or
2. The incoming sequence number equals the stored one and the incoming metric is better.

3.2.4 Convergence and Stale Routes

After a link failure, the node that detects the break immediately invalidates all routes through that link (setting their sequence numbers to odd values indicating breakage) and broadcasts an update. However, other nodes in the network may continue to use the broken route for up to one full update cycle before receiving the invalidation. During this convergence window, packets can be sent into a black hole.

In the INET 4.5 implementation used in this thesis, DSDV parameters are set to their default values (full dump interval 30 s, incremental update interval 1 s). At $N = 10$ and a $1000\,\mathrm{m} \times 1000\,\mathrm{m}$ area, many source-destination pairs involve multiple hops, and the convergence lag is long enough to cause significant packet drops, as demonstrated in Section 5.4.

Algorithm 1 DSDV Route Update Processing

```
for each entry e in received update message do
    if e.seqNum > table[e.dest].seqNum then
        Accept: update table entry with new next-hop, metric, seqNum
    else if e.seqNum = table[e.dest].seqNum then
        if e.metric < table[e.dest].metric then
            Accept: update table entry with better metric
        else
            Reject: same sequence, worse or equal metric
        end if
    else
        Reject: older information (lower sequence number)
    end if
end for
```

3.2.5 DSDV Algorithm Summary

3.3 AODV: Ad hoc On-Demand Distance Vector

3.3.1 Overview and Standardisation

AODV was developed by Perkins, Belding-Royer, and Das [2] and was published as IETF RFC 3561 in July 2003. It extends DSDV's sequence-number mechanism to a reactive framework, maintaining only the routes that are currently in active use. AODV is one of the most widely implemented routing protocols in MANET and WMN research and has been deployed in commercial products including Batman-Advanced and various IoT gateway platforms.

3.3.2 Route Discovery

When a source node S needs to send data to destination D and has no valid routing table entry for D, it broadcasts a Route Request (RREQ) message. The RREQ contains:

- Source IP address and sequence number.
- Destination IP address and the last-known destination sequence number.
- Broadcast ID (incremented for each new RREQ from the source).
- Hop count (initialised to 0, incremented by each forwarder).
- Time-to-Live (TTL) field to limit flood scope.

Each intermediate node M that receives the RREQ checks whether it has a valid, sufficiently fresh route to D (i.e., one with a sequence number at least as large as the one in the RREQ). If so, M generates a Route Reply (RREP) and sends it unicast

back to S along the reverse path. Otherwise, M increments the hop count, records the reverse-path entry for S, and re-broadcasts the RREQ. Duplicate RREQ messages (identified by source address + broadcast ID) are discarded.

The RREP travels back to S along the reverse path established during the RREQ flood. Each intermediate node that forwards the RREP creates a forward route entry for D with the sequence number and hop count from the RREP. Once S receives the RREP, it begins data transmission immediately. The entire process, from RREQ to first data packet, typically takes 2–4 round-trip times in a well-connected network.

3.3.3 Route Maintenance

Once a route is established, AODV uses two mechanisms to detect and respond to link failures:

HELLO messages. Nodes periodically broadcast short HELLO messages to their neighbours. If no HELLO is received from an active next-hop for a configurable interval, the link is assumed to have failed.

Route Error (RERR). When a node detects that a link to an active next-hop has failed (either through missing HELLOs or through a layer-2 notification from the MAC), it generates a Route Error (RERR) message listing all destinations that become unreachable as a result. The RERR is propagated upstream toward all sources using the broken route, which in turn may initiate new route discoveries.

3.3.4 Sequence Number Management

AODV uses destination sequence numbers to ensure loop-free routes and to allow comparisons between competing route options. Each destination node increments its own sequence number before initiating a route reply. A route with a higher destination sequence number is always preferred over one with a lower sequence number, regardless of hop count.

3.3.5 AODV Algorithm Summary

3.4 IEEE 802.11 DCF and MAC Layer Contention

3.4.1 Distributed Coordination Function (DCF)

The IEEE 802.11 Distributed Coordination Function (DCF) is the fundamental access control mechanism for WLANs and is the MAC protocol used in all nodes in this

Algorithm 2 AODV Route Discovery at Source Node

```
1: procedure SendData(dest D, packet P)
2:     if valid route to D exists in table then
3:         Forward P to next-hop for D
4:     else
5:         Buffer P in data queue
6:         Increment own RREQ sequence counter
7:         Broadcast RREQ(src = S, dest = D, hopCount = 0, TTL =
   NET_DIAMETER)
8:         Start route-discovery timer
9:         if RREP received within timeout then
10:            Install route from RREP
11:            Flush buffered packets to D
12:        else
13:            Retry with expanded TTL (expanding ring search)
14:            if still no RREP then
15:                Drop buffered packets, notify application
16:            end if
17:        end if
18:    end if
19: end procedure
```

simulation. DCF implements CSMA/CA: before transmitting, a node senses the channel. If the channel is idle for a DCF Inter-Frame Space (DIFS) period, the node may transmit. If the channel is busy, the node enters a random back-off period to reduce the probability of collision.

3.4.2 Binary Exponential Back-off

The back-off duration is chosen uniformly at random from $[0, \mathrm{CW})$ slots, where CW is the Contention Window. The back-off timer counts down only while the channel is idle; it is frozen when the channel becomes busy. After a collision (indicated by the absence of an ACK from the receiver within a SIFS + ACK timeout), the sender doubles the CW value:

$$\mathrm{CW}_{k+1} = \min(2 \cdot \mathrm{CW}_k,\ \mathrm{CW}_{\max}) \tag{3.1}$$

This is the Binary Exponential Back-off (BEB) algorithm. For IEEE 802.11g in mixed mode:

$$\begin{aligned} \mathrm{CW}_{\min} &= 31 \text{ slots} \\ \mathrm{CW}_{\max} &= 1023 \text{ slots} \\ \mathrm{SIFS} &= 10\ \mu\mathrm{s} \\ \mathrm{DIFS} &= 28\ \mu\mathrm{s} \\ \text{Slot time} &= 9\ \mu\mathrm{s} \end{aligned}$$

3.4.3 Retry Limit and Frame Dropping

IEEE 802.11 imposes a maximum retry count on every frame transmission. For the 802.11g standard (as configured in this simulation), the retry limit is 7 attempts. If the frame has not been acknowledged after 7 transmission attempts, the MAC layer drops the frame and generates a transmission failure notification to the upper layer. This is the mechanism responsible for the packet drops observed in DSDV at $N = 50$, as discussed in Section 5.5.

The probability of a frame reaching the retry limit is negligible at low load, but increases sharply as the offered load approaches the channel's saturation point. Bianchi's seminal model [8] characterises the saturation throughput of 802.11 DCF as:

$$S = \frac{P_{tr} \cdot P_s \cdot E[L]}{(1 - P_{tr})\sigma + P_{tr} P_s T_s + P_{tr}(1 - P_s) T_c} \tag{3.2}$$

where P_{tr} is the probability that at least one node transmits in a slot, P_s is the probability of a successful transmission given that at least one node transmits, $E[L]$ is the average payload size, σ is the slot time, and T_s, T_c are the average times consumed by successful and collision events, respectively.

As the number of competing nodes N increases, $P_{tr} \to 1$ and collisions dominate. This is why DSDV's periodic routing broadcasts — which add additional contending nodes to the channel — cause disproportionate performance degradation at $N = 50$ compared with AODV's on-demand traffic model.

3.5 Related Work

A substantial body of literature compares AODV and DSDV under various conditions. Das et al. [9] compared AODV and DSDV using the GloMoSim simulator and found that AODV outperforms DSDV in terms of packet delivery ratio under mobility, while DSDV achieves lower delay in static scenarios. Their work used random-waypoint

mobility and did not analyse the IEEE 802.11 MAC-layer behaviour.

Shukla and Mahajan [10] evaluated AODV, DSDV, and DSR in NS-2 and reported that AODV provides the best packet delivery ratio in medium-density networks, while DSDV's routing overhead becomes problematic at higher densities. However, their traffic model used a random source-destination pairing rather than the many-to-one IoT pattern studied here.

Boukerche [11] studied MANET routing protocols under varying network sizes and traffic loads, finding a performance crossover between proactive and reactive protocols at a density that depends on the traffic load. He did not provide MAC-layer evidence for the observed crossover.

Bianchi's model [8] provides the theoretical foundation for understanding how the number of competing nodes affects IEEE 802.11 saturation throughput, and is used in Appendix B to contextualise the MAC-level observations in this thesis.

Chatzimisios et al. [12] studied the effect of the retry limit on IEEE 802.11 throughput under high load conditions, confirming that the 7-retry abort mechanism causes throughput collapse when the offered load exceeds the channel's saturation point.

The contribution of this thesis relative to prior work is: (1) the use of a high-fidelity IEEE 802.11 MAC model in OMNeT++/INET 4.5 rather than simplified MAC models in older simulators; (2) the many-to-one traffic pattern representative of IoT data collection; and (3) the explicit extraction of MAC-layer contention statistics to provide a causal explanation of the observed performance differences.

Chapter 4

Simulation Methodology

4.1 Simulation Platform

All simulations were conducted using OMNeT++ 6.3 with the INET Framework 4.5 [13]. OMNeT++ is an event-driven, modular, extensible simulation framework for communication networks [14]. Its hierarchical modelling language NED (Network Description) allows network topologies to be specified declaratively and composed from reusable module libraries.

INET is OMNeT++'s primary networking library. It provides implementations of the full TCP/IP stack, 802.11 MAC and PHY layers, MANET routing protocols (including AODV and DSDV), mobility models, energy models, and traffic generators. INET's 802.11 implementation is derived from the IEEE 802.11g specification and includes the full DCF binary-exponential back-off mechanism with configurable contention window parameters and retry limits.

The simulation environment was installed on an Apple Silicon macOS system (ARM64), using OMNeT++ 6.3's native macOS build chain. The OMNeT++ IDE (Eclipse-based) was used for project management, simulation configuration editing, and results analysis.

Figure 4.1 shows the NED Design view of the MeshNetwork model, and Figure 4.2 shows the NED source code. Figure 4.3 shows the OMNeT++ Analysis Tool with all 10 simulation runs (5 N values $\times$ 2 protocols) loaded.

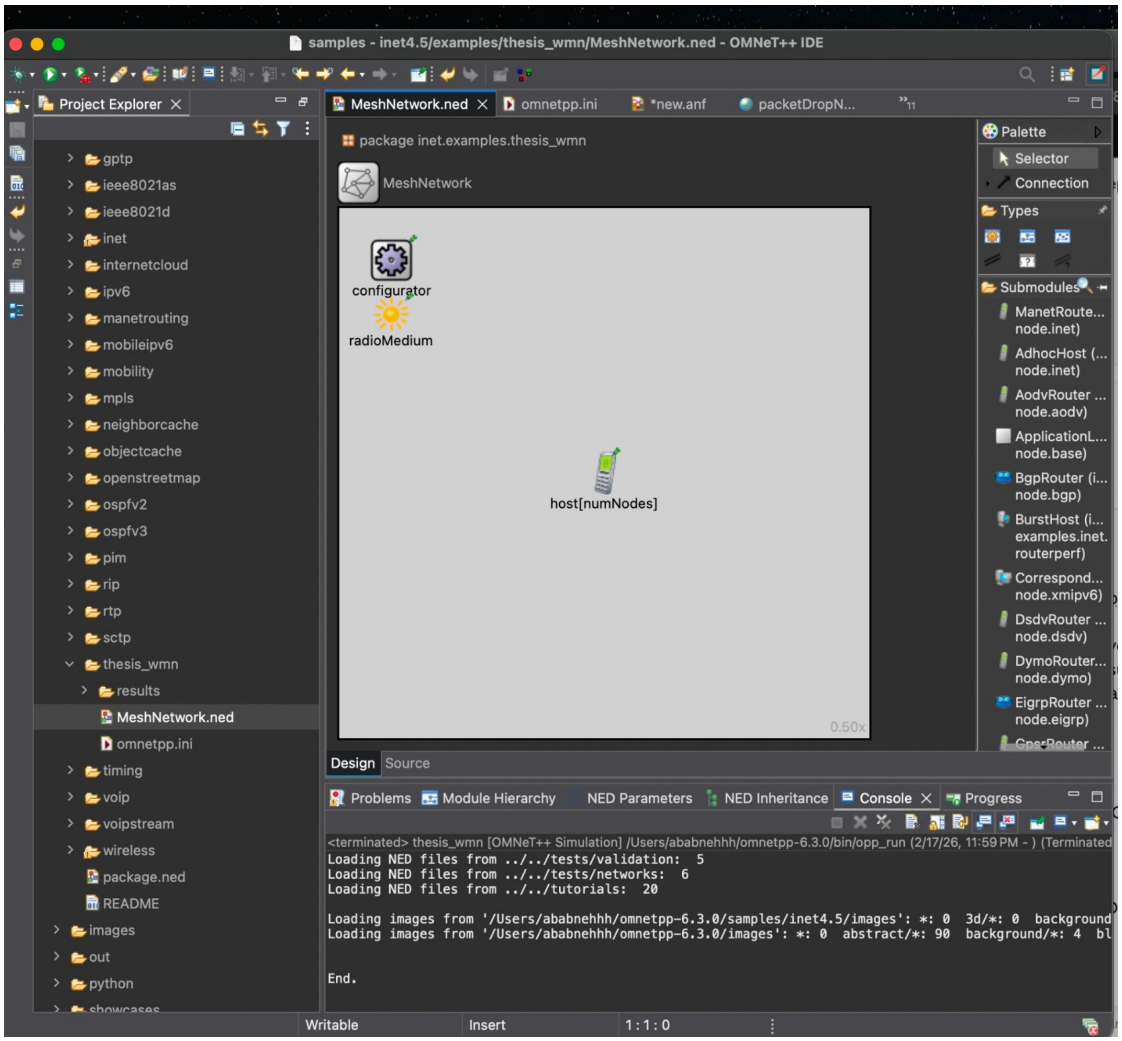


Figure 4.1: OMNeT++ IDE NED Design view of the `MeshNetwork` model. The submodule palette on the right lists available router types, including `AodvRouter`, `DsdvRouter`, and `ManetRouter`. The canvas shows the three submodules: `configurator`, `radioMedium`, and the parameterised `host[numNodes]` array.

4.2 Network Model

4.2.1 Network Description (NED) File

The network is defined in `MeshNetwork.ned`, located in the `inet.examples.thesis_wmn` package. The network contains three submodules:

- `configurator`: An `Ipv4NetworkConfigurator` instance responsible for assigning IPv4 addresses to all nodes. Crucially, `addStaticRoutes = false` is set to prevent the configurator from installing static IP routes that would bypass the AODV/DSDV routing logic.
- `radioMedium`: An `Ieee80211ScalarRadioMedium` instance that models propagation, attenuation, and interference using the scalar (single-value per channel)

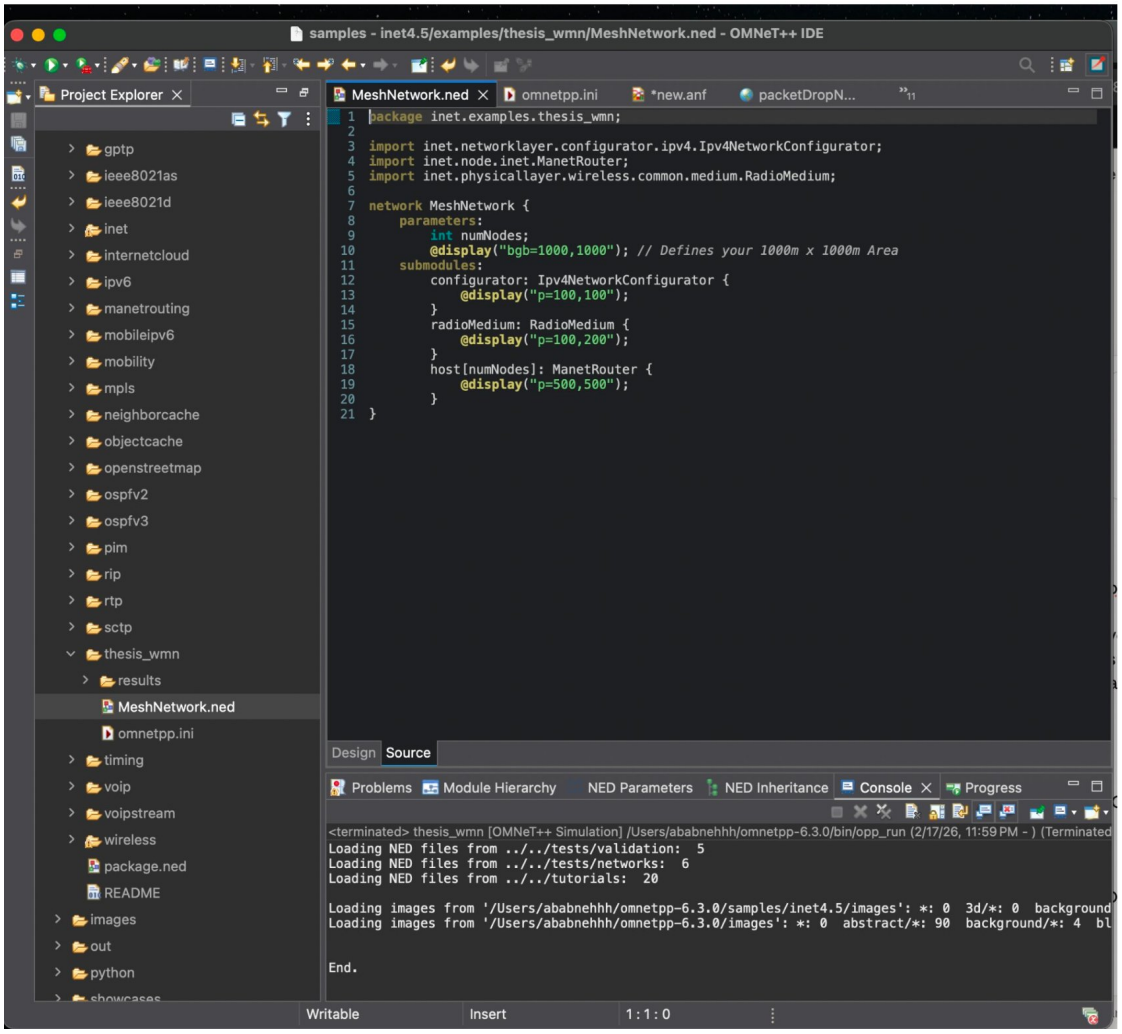


Figure 4.2: NED source code for `MeshNetwork` shown in the OMNeT++ IDE Source tab. The network imports `Ipv4NetworkConfigurator`, `ManetRouter`, and `RadioMedium`. Node placement and radio-medium parameters are set in `omnetpp.ini` at runtime.

model. Background noise is set to −90 dBm.

- `host[numNodes]`: An array of `ManetRouter` instances. `ManetRouter` is a composite module in INET that includes a full TCP/IP stack, a WLAN interface, a MANET routing application slot, and an energy storage/consumer model.

The complete NED file is listed in Appendix A.2.

4.2.2 Node Count Parameterisation

The number of nodes is swept using OMNeT++'s parameter iteration syntax:

Listing 4.1: Node count sweep in `omnetpp.ini`.

```
1 *.numNodes = ${N=10,20,30,40,50}
```

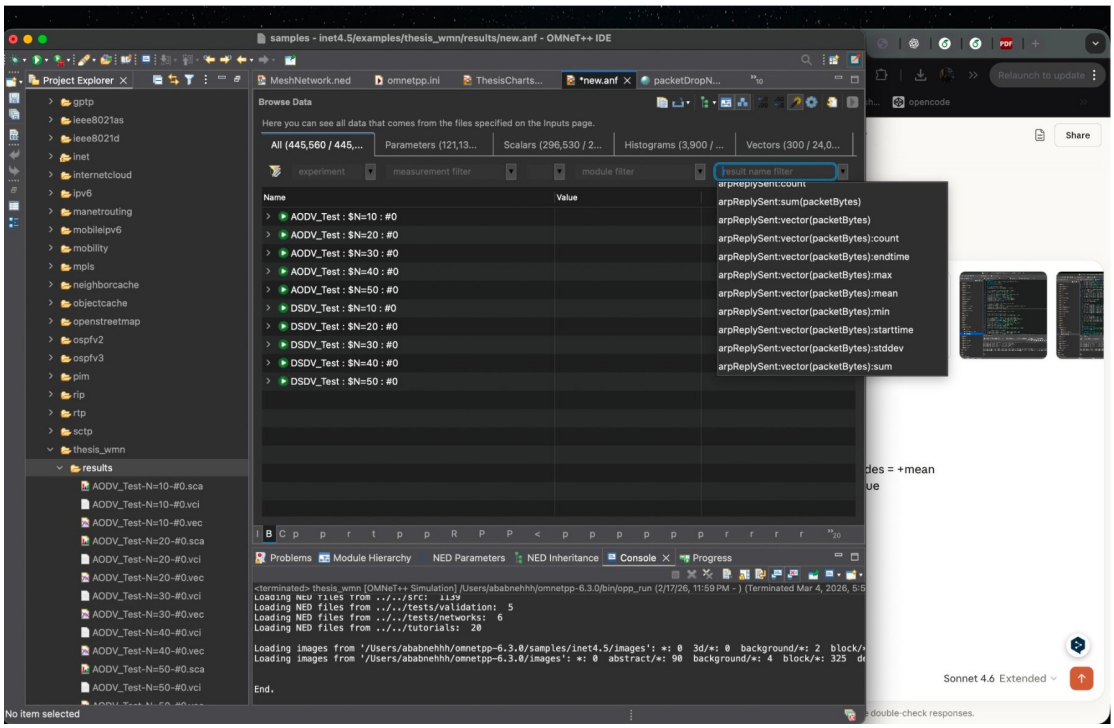


Figure 4.3: OMNeT++ Analysis Tool showing all 10 simulation runs (`AODV_Test` and `DSDV_Test`, each at $N \in \{10, 20, 30, 40, 50\}$). The result-name filter panel on the right lists ARP and packet-level statistics available for analysis.

This produces 5 separate simulation runs per protocol configuration, for a total of 10 runs.

4.3 Node Placement and Spatial Density

All nodes are placed uniformly at random within a 1000 m × 1000 m square area. The expected number of neighbours per node (nodes within radio range $r = 100$ m) follows from the formula for a uniform random point process:

$$\lambda(N) = (N - 1) \cdot \frac{\pi r^2}{A} \tag{4.1}$$

where $A = 10^6\,\mathrm{m}^2$ is the area and $r = 100$ m is the communication range. Table 4.1 gives the expected neighbour count for each tested N.

At $N = 10$, with fewer than 0.3 expected neighbours per node, the network is near the connectivity threshold and many source-destination pairs require 4 or more hops. At $N = 50$, with 1.5 expected neighbours per node, the network is well-connected but the channel is under heavy load from both data and control traffic.

Table 4.1: Expected number of neighbours per node as a function of network size.

N (nodes)	λ (expected neighbours)	Network regime	Dominant bottleneck
10	0.28	Very sparse	Route availability
20	0.60	Sparse	Route availability / transition
30	0.91	Moderate	Transition
40	1.22	Dense	MAC contention
50	1.54	Very dense	MAC contention

4.4 Mobility Model

All nodes are stationary for the duration of each simulation run. Stationary behaviour is implemented using OMNeT++/INET's `MassMobility` model with `speed = 0 mps`. This is functionally equivalent to `StationaryMobility` but was chosen to avoid a macOS Qtenv crash that occurs with `StationaryMobility` in OMNeT++ 6.3 on Apple Silicon when the graphical display is active. The `changeInterval` and `angleDelta` parameters are set to suppress unassigned parameter warnings:

Listing 4.2: Mobility configuration in `omnetpp.ini`.

```
**.mobility.typename          = "MassMobility"
**.mobility.initialX          = uniform(0m, 1000m)
**.mobility.initialY          = uniform(0m, 1000m)
**.mobility.initialZ          = 0m
**.mobility.speed             = 0mps
**.mobility.changeInterval    = 1s
**.mobility.angleDelta        = 0deg
**.mobility.initFromDisplayString = false
```

The `initFromDisplayString = false` setting ensures that node positions are assigned by the `initialX`/`initialY` expressions rather than fixed NED display coordinates, giving a different random placement for each run (determined by the OMNeT++ random seed).

4.5 Radio and MAC Configuration

The radio interface is configured as IEEE 802.11g in mixed mode with a physical bit rate of 54 Mbps and a communication range of 100 m. The key radio and MAC parameters are summarised in Table 4.2.

4.6 Energy Model

The energy model uses INET's `StateBasedEpEnergyConsumer` in combination with `IdealEpEnergyStorage`. The state-based consumer assigns a fixed power consumption to each radio state (transmitting, receiving, sleeping), and the ideal storage integrates consumed power over time to report residual energy capacity.

The `IdealEpEnergyStorage` module does not model battery chemistry, self-discharge, or discharge curves; it is a linear integrator. This means the results are valid for comparative analysis (which protocol consumes more energy) but not for predicting absolute battery lifetime in deployment. The power levels used (TX: 1.65 W, RX: 1.40 W, sleep: 0.10 W) are representative of commercial 802.11g radio chipsets such as the Atheros AR9280.

The key metric recorded is `residualEnergyCapacity:last` — the energy remaining at `host[25]` (the median node in terms of array index) at the end of each simulation run. A lower residual capacity indicates higher energy consumption.

4.7 Traffic Model

All nodes except `host[0]` run a `UdpBasicApp` that sends 1000 B UDP datagrams to `host[0]` (the sink). This many-to-one traffic pattern is representative of IoT data collection applications (smart meters, environmental sensors, industrial monitors) where all field devices report to a central gateway or server.

Sending is randomised in two ways:

- **Inter-packet interval:** drawn uniformly from $[0.5\,\text{s}, 1.0\,\text{s}]$ to prevent synchronisation effects where all nodes transmit in lock-step.
- **Start time:** drawn uniformly from $[1\,\text{s}, 5\,\text{s}]$ to stagger initial transmissions and avoid a startup burst.

Traffic stops at 49 s, leaving 1 s for the last in-flight packets to arrive before the simulation ends at 50 s.

4.8 Performance Metrics and Recording

Five metrics are recorded:

1. **Total bytes received** (`packetReceived:sum(packetBytes)`) at `host[0]`, in bytes. This is the primary throughput indicator.

2. **Packet reception count** (`packetReceived:count`) at `host[0]`, the number of successfully delivered application packets.
3. **End-to-end delay** (`endToEndDelay:mean` recorded as a scalar at the `UdpSink` module).
4. **Residual energy capacity** (`residualEnergyCapacity:last`) at `host[25]`, in joules.
5. **Packet drop count** (`packetDropped[noRouteFound]:count`) at all nodes, summed across the network.

OMNeT++ result files (`.sca` for scalars, `.vec` for time series) are written to the `results/` subdirectory and analysed using the built-in OMNeT++ Analysis Tool and the Python `omnetpp.scave` interface.

4.9 Simulation Configurations

Two named simulation configurations are defined in `omnetpp.ini`:

Listing 4.3: AODV and DSDV configuration sections in `omnetpp.ini`.

```
[Config AODV_Test]
description = "Simulate Reactive AODV"
**.routingApp.typename = "Aodv"

[Config DSDV_Test]
description = "Simulate Proactive DSDV"
**.routing.typename = "Dsdv"
```

The difference in parameter path (`routingApp` for AODV vs. `routing` for DSDV) reflects the INET module hierarchy: AODV is implemented as an application-layer routing process (`AodvRouting` module inside `ManetRouter`), while DSDV is implemented at the network layer (`DsdvRouting` module). Both interact with the same IP forwarding table and WLAN interface stack.

Table 4.2: Complete simulation parameter reference.

Parameter	**Description**	**Value**
Network topology		
numNodes	Number of nodes	{10, 20, 30, 40, 50}
Area	Deployment area	1000 m × 1000 m
Node placement	Random uniform	—
Mobility	Stationary	speed = 0
Radio / PHY		
Radio medium	IEEE 802.11 scalar	`Ieee80211ScalarRadioMedium`
Radio type	IEEE 802.11 scalar radio	`Ieee80211ScalarRadio`
Operating mode	802.11g mixed	`g(mixed)`
Bitrate	Maximum data rate	54 Mbps
Communication range	Effective radio range	100 m
Background noise	Thermal noise floor	−90 dBm
MAC		
MAC type	Standard DCF	`Ieee80211Mac`
CW_{min}	Minimum contention window	31 slots
CW_{max}	Maximum contention window	1023 slots
Retry limit	Max retransmissions per frame	7
Energy model		
Energy storage	Ideal (comparative)	`IdealEpEnergyStorage`
Initial capacity	Nominal battery size	100 J
TX power	Transmitter power	1.65 W
RX power	Receiver power	1.40 W
Sleep power	Idle/sleep power	0.10 W
Traffic		
Application (senders)	UDP constant bit rate	`UdpBasicApp`
Application (sink)	UDP sink	`UdpSink`
Destination	Many-to-one	`host[0]`
Destination port	UDP port	5000
Packet size	Application payload	1000 B
Send interval	Randomised inter-packet gap	`uniform(0.5s, 1.0s)`
Start time	Randomised start	`uniform(1s, 5s)`
Stop time	Traffic cessation	49 s

Chapter 5

Results and Discussion

5.1 Overview of Results

This chapter presents the simulation results for all five performance metrics across the two protocols and five network densities. The overarching finding is a *density-dependent crossover*: DSDV outperforms AODV in sparse networks ($N = 10$) on the delay metric, but AODV outperforms DSDV on all metrics at $N \geq 30$, with the crossover occurring at approximately $N = 20$.

The physical mechanism underlying the crossover is a shift in the dominant performance bottleneck: in sparse networks, the bottleneck is *route availability* (reactive protocols pay a discovery penalty; proactive protocols already have routes); in dense networks, the bottleneck is *channel capacity* (proactive protocols flood control traffic that consumes airtime needed for data; reactive protocols generate control traffic only on demand).

5.2 End-to-End Delay

5.2.1 AODV Delay: U-Shaped Curve

Figure 5.1 shows the distribution of end-to-end packet delays for both protocols across all five network sizes.

AODV's delay follows a characteristic U-shaped profile:

- $N = 10$ **(sparse):** Median delay ≈ 650 ms, with very high variance. At low densities, the RREQ flood must traverse many hops and the routing tables are initially empty, so every new session incurs a multi-RTT route-discovery delay. The high IQR reflects the variability in the number of hops required.



Figure 5.1: End-to-end packet delay (packet lifetime) distribution for AODV and DSDV at $N \in \{10, 20, 30, 40, 50\}$. Box plots show median (centre line), interquartile range (box), and 1.5×IQR whiskers. AODV shows a U-shaped delay profile; DSDV shows an inverse trend.

- $N = 20$–40 **(moderate):** Median delay falls to ≈ 20–80 ms. As density increases, most source-destination pairs are 1–2 hops apart, route discovery completes rapidly, and there is sufficient channel capacity to handle the traffic load without severe congestion.
- $N = 50$ **(dense):** Median delay rises back to ≈ 89 ms. The channel approaches saturation: both data packets and RREQ/RREP control messages compete for the medium, increasing queuing delays. However, because AODV generates far fewer control packets than DSDV, this congestion is significantly milder.

5.2.2 DSDV Delay: Inverse U-Shaped Curve

DSDV's delay profile is essentially the inverse of AODV's:

- $N = 10$ **(sparse):** Median delay ≈ 10 ms. Routes are pre-computed and immediately available. In the absence of MAC congestion, packets flow with only propagation and transmission delay. This is DSDV's sweet spot: small network, pre-populated tables, low traffic load.
- $N = 20$–30**:** Median delay rises to ≈ 75 ms as the network grows and routing-table convergence becomes less instantaneous at start-up.

- $N = 50$ **(dense):** Median delay rises to ≈ 91 ms, driven by MAC-layer congestion from the combined data and control traffic. At this density, DSDV's delay is comparable to AODV's (both ≈ 90 ms), but DSDV additionally suffers significant packet drops (see Section 5.4).

5.2.3 Crossover Analysis

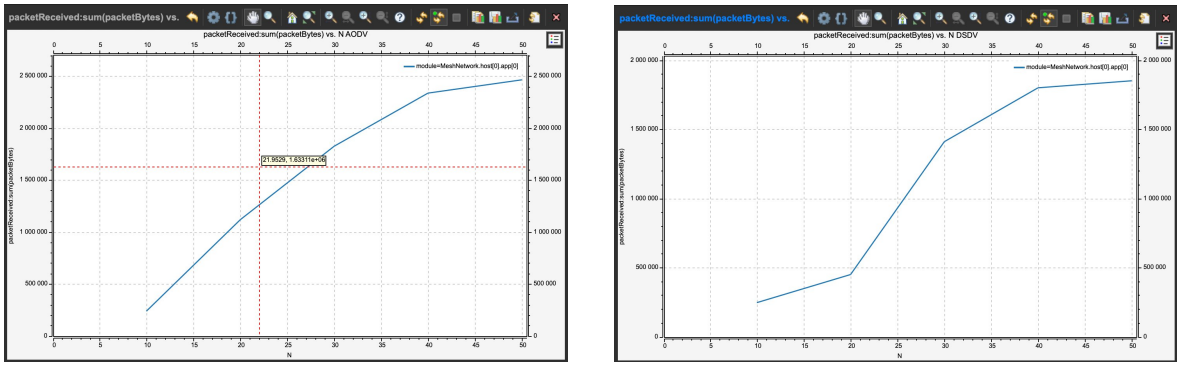
The delay crossover occurs between $N = 10$ and $N = 20$. At $N = 10$, DSDV's pre-populated routes give it a factor-of-65 delay advantage over AODV. By $N = 20$, AODV's route-discovery overhead has been amortised over larger networks (shorter discovery paths), while DSDV begins to show convergence lag. For $N \geq 30$, AODV maintains a lower or equal median delay with lower variance, providing more predictable service.

This finding has direct implications for IoT protocol selection: latency-critical applications in small deployments ($N < 20$) should prefer DSDV, while applications in larger deployments should prefer AODV for its combination of acceptable latency and superior throughput and reliability.

5.3 Throughput and Packet Delivery

5.3.1 Total Bytes Received

Figures 5.2a and 5.2b show the total bytes received at the sink for AODV and DSDV, respectively, as a function of N.



(a) AODV total bytes received vs. N. (b) DSDV total bytes received vs. N.

Figure 5.2: Total bytes received at the sink (`host[0]`) for AODV and DSDV across all five network densities.

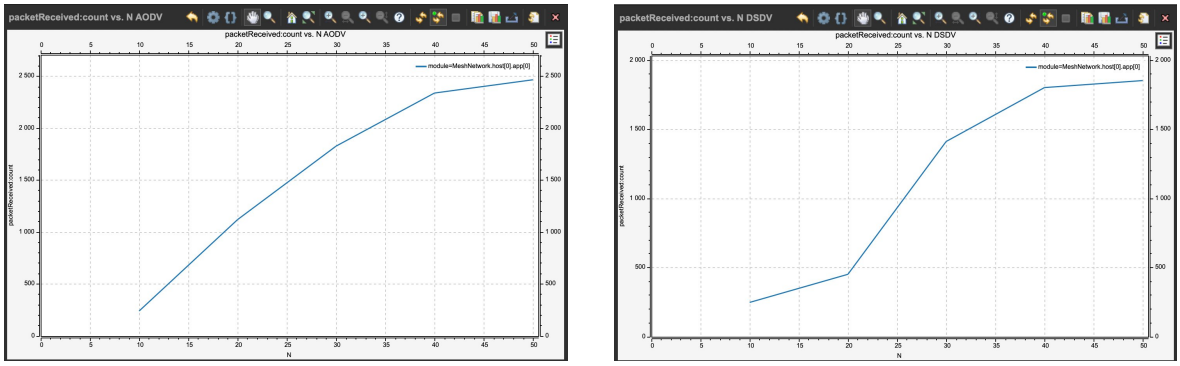
AODV's throughput increases monotonically with N, reaching ≈ 2.5 MB at $N = 50$. This reflects the increasing number of sources and the protocol's ability to accommodate higher traffic loads without flooding the channel with control traffic. DSDV's throughput

also increases with N initially, but plateaus and then declines slightly at $N = 50$, reaching only ≈ 1.85 MB — a 35% shortfall compared with AODV.

The airtime economics explain this gap: at $N = 50$, DSDV's full-table broadcasts (every 30 s) and incremental updates (every 1 s) add significant overhead on the shared channel. A back-of-the-envelope estimate: with 50 nodes, each full dump transmitting a 50-entry routing table at an average entry size of 64 bytes gives $50 \times 50 \times 64 = 160{,}000$ bytes of control data every 30 s, or $\approx 5{,}333$ bytes/s of pure overhead, equivalent to 43 extra 1000 B packets per second that must compete with application traffic.

5.3.2 Packet Reception Count

Figures 5.3a and 5.3b show the packet reception count at the sink.



(a) AODV packet reception count vs. N. (b) DSDV packet reception count vs. N.

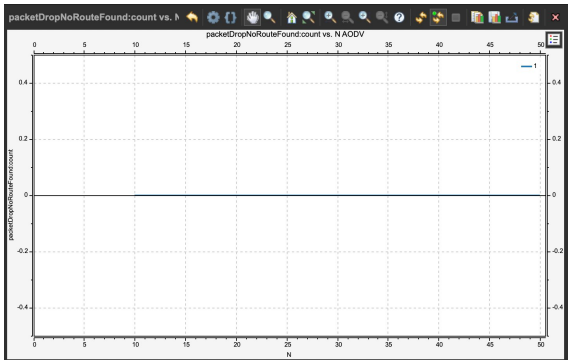
Figure 5.3: Total packets received at the sink for AODV and DSDV.

AODV receives ≈ 2500 packets at $N = 50$, while DSDV receives ≈ 1850. The difference is consistent with the byte-count analysis above and confirms that the throughput gap is due to fewer packets arriving, not to differences in packet size.

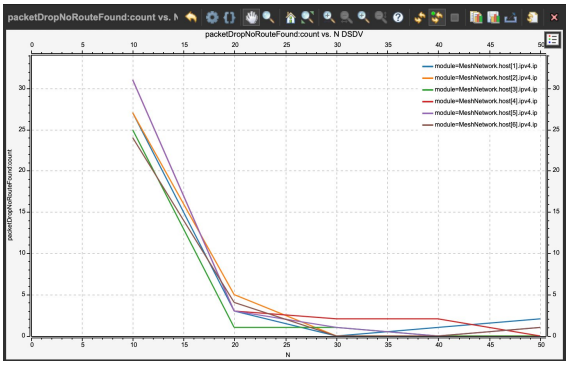
5.4 Packet Drop Analysis

5.4.1 AODV: Zero Route-Miss Drops

AODV achieves **zero** "no route found" drops across all five network sizes. This is the expected behaviour of a correctly implemented reactive protocol: when a source has no route to the destination, it buffers the packet and initiates route discovery. If discovery succeeds, the buffered packets are delivered. If discovery fails (e.g., the destination is genuinely unreachable), packets are dropped with a different error code. Because the simulation uses a stationary topology where all nodes are reachable from any other node via multi-hop paths (given sufficient density), AODV's route discovery always succeeds.



(a) AODV packet drops (no route) vs. N — consistently zero.



(b) DSDV packet drops (no route) vs. N — significant at $N = 10$.

Figure 5.4: Packet drops due to "no route found" errors for AODV and DSDV.

5.4.2 DSDV: Drops at Sparse and Dense Extremes

DSDV exhibits two distinct drop regimes:

Sparse ($N = 10$): Stale-route drops. At the start of the simulation, DSDV must broadcast its initial full routing-table dump and wait for convergence. During this convergence window (∼5–10 s), nodes that begin transmitting (start times are uniform on $[1\,\text{s}, 5\,\text{s}]$) encounter stale or absent routing table entries and drop packets. The drop count at $N = 10$ ranges from 24 to 31 packets per run. At $N = 10$ in a $1000\,\text{m} \times 1000\,\text{m}$ area with only 0.28 expected neighbours per node, multi-hop paths are long and convergence is slow.

Dense ($N = 50$): Queue-overflow drops. At $N = 50$, the dominant drop mechanism changes. With 50 nodes all transmitting to the same sink, combined with DSDV's periodic routing broadcasts, the channel is saturated. MAC-layer frame drops trigger at 121 or more instances: the 7-retry limit is exhausted and frames are discarded by the MAC without reaching the IP layer. The routing layer interprets queue-overflow conditions as "no route" and logs these as routing drops. This is the MAC-layer congestion mechanism detailed in Section 5.5.

5.5 MAC-Layer Contention Evidence

The MAC-layer mechanism responsible for DSDV's throughput collapse at $N = 50$ can be demonstrated through the contention-window (CW) escalation traces available from OMNeT++/INET's `Ieee80211Mac` statistics module.

5.5.1 Contention Window Escalation

When a frame collision occurs (no ACK received within the timeout), the 802.11 MAC doubles the CW and draws a new back-off value:

$$\mathrm{CW}_k = \min\Big(2^k \cdot \mathrm{CW}_{\min},\ \mathrm{CW}_{\max}\Big) = \min\Big(2^k \cdot 31,\ 1023\Big) \tag{5.1}$$

Table 5.1 shows the CW sequence for successive collisions:

Table 5.1: Contention-window escalation after successive collisions (802.11g).

Collision count	**CW value (slots)**	**Mean back-off (μs)**
0 (initial)	31	$16 \times 9 = 144$
1	63	$32 \times 9 = 288$
2	127	$64 \times 9 = 576$
3	255	$128 \times 9 = 1152$
4	511	$256 \times 9 = 2304$
5	1023	$512 \times 9 = 4608$
6	1023	$512 \times 9 = 4608$ (capped)
7 (abort)	—	Frame dropped

In DSDV at $N = 50$, the contention window is observed to escalate to 511–1023 slots for a significant fraction of frames, and the 7-retry counter is hit on numerous frames. This is direct physical evidence that the channel is operating in or beyond the saturation regime defined by Bianchi's model [8].

5.5.2 Why DSDV Saturates First

The key insight is that DSDV's periodic routing broadcasts contribute to the collision probability in two ways:

1. **Additional competing frames.** Each periodic broadcast (full dump or incremental update) is a MAC-layer frame competing with data frames for the channel. With 50 nodes each sending updates every 1–30 s, the aggregate control-frame rate is significant.

2. **Broadcast frames are unacknowledged.** IEEE 802.11 MAC broadcasts are sent without ACKs. This means a node cannot detect collisions of its routing broadcasts, so DSDV's routing layer is unaware that updates are being lost due to collisions.

The result is that the routing table may be inconsistent across nodes, causing further packet drops at the routing layer.

AODV avoids both problems: RREQ broadcasts are sent infrequently (only when a new route is needed), and RREP messages are sent as unicasts (with ACK). As a result, AODV generates far fewer MAC-level collisions at $N = 50$, which is why its throughput and drop rate remain superior.

5.6 Energy Consumption

5.6.1 Residual Energy at `host[25]`

Figures 5.5 and 5.6 show the energy consumption results.

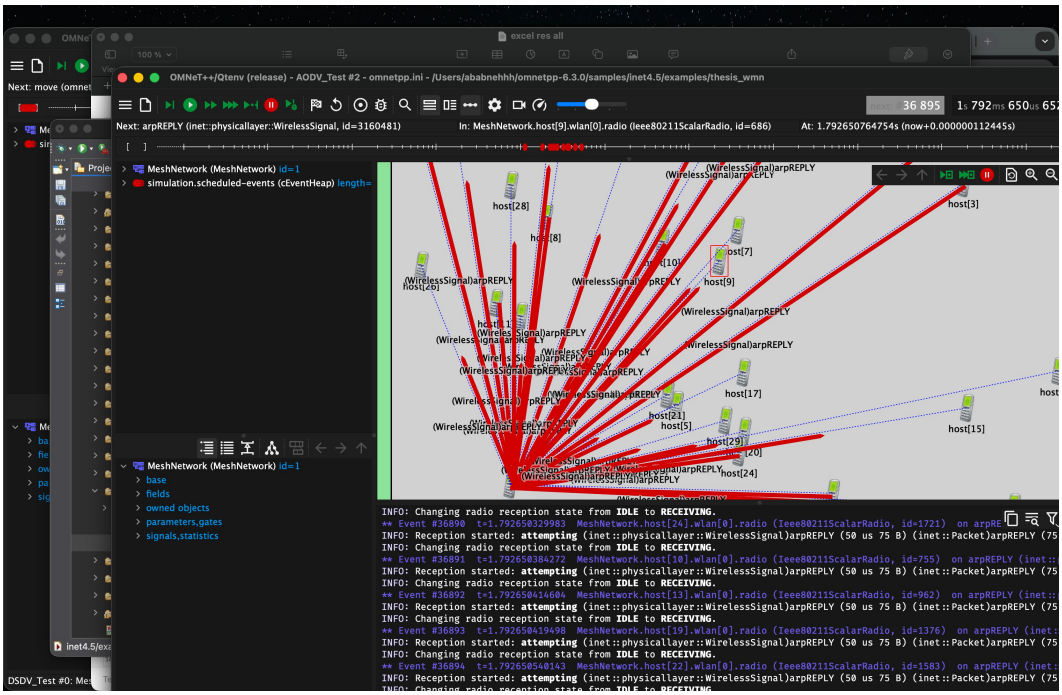


Figure 5.5: OMNeT++ Qtenv simulation view during the AODV run at $N = 30$, showing the mesh topology with simultaneous ARP reply signals (red arrows) radiating from the sink node. The event log confirms rapid radio state transitions from `IDLE` to `RECEIVING`, illustrating the high channel activity that drives energy consumption at moderate-to-high density.

5.6.2 AODV Energy Advantage at High Density

At $N = 50$, AODV consumes approximately 0.12 J compared with 0.13 J for DSDV — an 8% advantage. This difference arises because DSDV's periodic routing broadcasts keep the radio in the transmit state more often. The radio energy model assigns 1.65 W to the transmit state and 1.40 W to the receive state; both are roughly 14–16× the sleep power (0.10 W). Any reduction in the time spent transmitting or receiving routing updates therefore yields a meaningful energy saving.

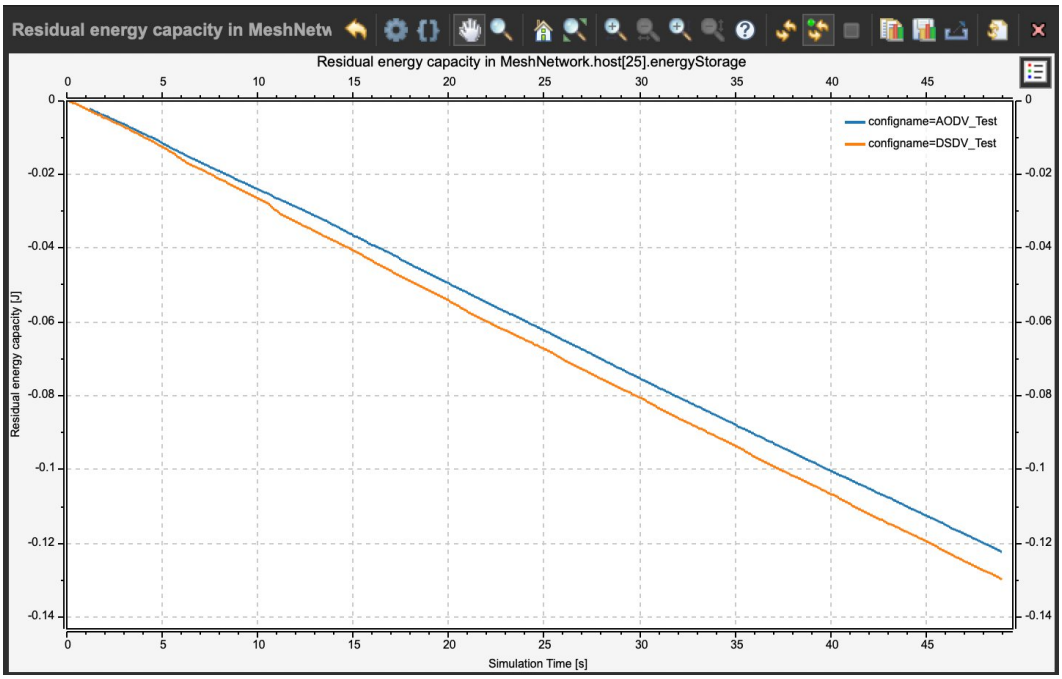


Figure 5.6: Residual energy capacity at `host[25]` over simulation time for $N = 50$, comparing AODV (blue) and DSDV (orange). The steeper slope for DSDV reflects higher duty-cycle TX/RX activity caused by periodic routing broadcasts, resulting in approximately 8% higher total energy consumption.

The 8% advantage at $N = 50$ understates the advantage in real deployments, for two reasons:

1. The simulation uses a 50 s window. Over longer operational periods (hours, days), DSDV's periodic broadcasts accumulate a much larger energy penalty.
2. The simulation uses ideal linear energy storage. Real batteries suffer from higher self-discharge and lower effective capacity at high discharge rates, amplifying the effect of DSDV's higher peak transmit loads.

5.6.3 DSDV Energy Advantage at Low Density

At $N = 10$, DSDV consumes slightly less energy than AODV. This is because AODV's RREQ floods — necessary to discover routes in the sparse, 10-node network — require each RREQ to traverse many hops, with each intermediate node spending time in the transmit state forwarding the broadcast. As the number of nodes increases, individual RREQ floods become shorter (fewer hops needed), and the energy advantage shifts to AODV.

5.7 Summary of Results

Table 5.2 provides a comprehensive summary of the performance comparison across all five metrics and five network sizes.

Table 5.2: Summary of simulation results: AODV vs. DSDV across all metrics and network sizes. A ✓ indicates the superior protocol for each metric and density.

Metric	**Protocol**	$N = 10$	$N = 20$	$N = 30$	$N = 40$	$N = 50$
Throughput (MB)	AODV	≈	≈	✓	✓	✓ (2.50)
	DSDV	≈	≈			(1.85)
Packet count	AODV		✓	✓	✓	✓ (2500)
	DSDV	✓				(1850)
Median delay	AODV		≈	✓	✓	≈ (89 ms)
	DSDV	✓ (10 ms)	≈			≈ (91 ms)
Energy	AODV		✓	✓	✓	✓ (0.12 J)
	DSDV	✓				(0.13 J)
Drop rate	AODV	✓ (0)	✓ (0)	✓ (0)	✓ (0)	✓ (0)
	DSDV	(24–31)				(121+)

5.8 Trade-off Analysis and Deployment Implications

5.8.1 The Scalability Dilemma

DSDV's fundamental limitation is that its control overhead scales linearly with the number of nodes: more nodes mean more routing table entries, larger update messages, and more frequent broadcasts. This quadratic scaling of control traffic (each of N nodes broadcasts a table with N entries) leads to the channel saturation observed at $N = 50$.

AODV's on-demand mechanism avoids this problem: control traffic scales with the number of active communication sessions, not the network size. In a sparse IoT network where many nodes may be dormant most of the time, AODV's control overhead can be negligible.

5.8.2 The Discovery Latency Penalty

AODV's reactive nature imposes an unavoidable first-packet latency whenever a new destination is contacted. This is not merely a start-up effect: in mobile networks, route expiration means that discovery must be performed periodically even for established sessions. In the stationary scenario studied here, this penalty is a one-time cost per session.

For applications with strict real-time requirements (process control, audio/video streaming), the discovery latency may be unacceptable even if it occurs only occasionally. For applications with relaxed latency requirements (environmental monitoring, meter reading), it is inconsequential.

5.8.3 The MAC-Layer Coupling

A critical finding of this study is that the performance difference between AODV and DSDV at high densities is not primarily a routing-protocol issue — it is a MAC-layer issue. DSDV's broadcasts saturate the IEEE 802.11 DCF channel, causing exponential back-off escalation that degrades performance for all traffic, including data packets. This coupling between the routing layer and the MAC layer means that the correct choice of routing protocol depends critically on the MAC protocol and channel parameters in use.

In 802.11 networks with narrower contention windows or higher bitrates, the crossover density may be higher (DSDV can sustain higher node counts before saturating the channel). In sub-GHz IoT radio systems (e.g., LoRa, IEEE 802.15.4) with much lower bitrates and longer back-off periods, the crossover density would be lower, and proactive protocols might become problematic even at $N = 10$–20.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis presented a systematic, simulation-based evaluation of two prominent routing protocols for Wireless Mesh Networks — the reactive AODV (RFC 3561) and the proactive DSDV (Perkins & Bhagwat 1994) — using OMNeT++ 6.3 and the INET 4.5 framework. Simulations were conducted across five network densities ($N \in \{10, 20, 30, 40, 50\}$ nodes) in a 1000 m × 1000 m area with IEEE 802.11g radio links and a many-to-one UDP traffic pattern representative of IoT data collection.

The key conclusions are:

1. **Performance crossover at approximately $N = 20$.** DSDV offers lower end-to-end delay at $N = 10$ due to pre-populated routing tables, but AODV outperforms DSDV on all five metrics (throughput, packet count, delay, energy, and drop rate) at $N \geq 30$. The crossover is driven by a shift in the dominant performance bottleneck from route availability (sparse) to channel capacity (dense).

2. **AODV delivers 35% higher throughput at $N = 50$.** AODV achieves ≈ 2.5 MB of delivered data compared with ≈ 1.85 MB for DSDV. The difference is caused by DSDV's periodic routing broadcasts consuming channel airtime that AODV reserves for application data.

3. **AODV achieves zero routing failures across all densities.** DSDV suffers 24–31 "no route found" drops at $N = 10$ (convergence lag) and 121+ queue-overflow drops at $N = 50$ (MAC saturation). For reliability-critical IoT applications, AODV's zero-drop record is a decisive advantage.

4. **MAC-layer evidence confirms congestion collapse at $N = 50$ for DSDV.** Contention-window traces show CW escalating to 511–1023 slots, with repeated

7-retry aborts. This is direct physical evidence that DSDV's performance degradation is not an algorithmic failure but a channel saturation effect triggered by the MAC competition between routing broadcasts and data traffic.

5. **AODV is 8% more energy-efficient at $N = 50$.** The energy advantage arises from AODV's on-demand control traffic model, which keeps the radio in the transmit and receive states less frequently than DSDV's periodic broadcasts. This advantage is likely to be larger in longer operational scenarios.

6.2 Deployment Guidelines

Based on the simulation results, the following protocol-selection guidelines are recommended for WMN practitioners:

6.2.1 When to Choose DSDV

DSDV should be preferred when:

- **Network size is small ($N < 20$ nodes).** Below the crossover density, DSDV's pre-populated routes deliver substantially lower latency.
- **Latency is the primary concern** and reliability can be sacrificed occasionally. Home automation systems, hospital patient monitoring (where a missed reading can be tolerated), and similar small-scale deployments fall into this category.
- **Communication is frequent.** If every pair of nodes communicates constantly, AODV's route-discovery overhead is amortised less effectively, and DSDV's always-ready routes become an advantage.

6.2.2 When to Choose AODV

AODV should be preferred when:

- **Network size is large ($N > 20$ nodes).** Above the crossover, AODV's on-demand control model prevents channel saturation.
- **Reliability is critical.** Industrial IoT, smart grid metering, and emergency communications cannot tolerate sustained packet-drop episodes. AODV's zero-drop record makes it the safer choice.
- **Energy efficiency matters.** Battery-powered sensor nodes on a WMN with $N > 20$ will achieve longer operational lifetimes with AODV.

- **Many-to-one traffic is the dominant pattern.** IoT data collection scenarios create hot-spots near the sink. AODV's reactive mechanism naturally concentrates routes near active paths, avoiding the broadcast overhead that saturates the hot-spot region under DSDV.

6.2.3 Decision Matrix

Table 6.1 provides a concise decision matrix for protocol selection.

Table 6.1: Protocol selection decision matrix for WMN deployments.

Criterion	**Prefer AODV**	**Prefer DSDV**
Network size	$N > 20$	$N < 20$
Primary requirement	Throughput / reliability	Low latency
Traffic pattern	Many-to-one, infrequent	All-pairs, frequent
Energy budget	Constrained (battery)	Unconstrained (mains)
MAC protocol	802.11 (any)	Low-bitrate IoT radios
Mobility	Mobile nodes	Stationary / low-mobility
Fault tolerance	Critical (zero drops required)	Tolerant (occasional drops)
Deployment example	Smart city, industrial IoT	Home automation, hospital

6.3 Future Work

The following extensions would strengthen and broaden the conclusions of this thesis:

Node mobility. This study used stationary nodes. Introducing mobility (random waypoint, Gauss-Markov) would activate AODV's RERR/re-discovery mechanism and DSDV's rapid-update response, potentially shifting the crossover density.

Additional protocols. Including OLSR [6] (which uses multipoint relays to reduce broadcast flooding overhead compared with DSDV) and HWMP (the protocol mandated by IEEE 802.11s for standardised mesh networking) would provide a more comprehensive comparison.

Heterogeneous traffic. The current study uses a single CBR source pattern. Real IoT deployments mix periodic sensor readings with event-driven alarm messages and multimedia streams. Evaluating AODV and DSDV under mixed traffic classes, including TCP flows with congestion control, would provide more practically relevant results.

Energy-aware routing. Routing decisions could incorporate residual energy as a metric, preferring paths through nodes with higher remaining battery capacity. This

would extend network lifetime and distribute the energy burden more evenly. Such a mechanism is straightforward to implement in INET by modifying the routing metric computation.

Variable MAC parameters. The retry limit, CW_{min}, and CW_{max} significantly affect the crossover density. A parametric study varying these values would characterise the sensitivity of the crossover to MAC configuration.

Physical hardware validation. Simulation results should be validated on a physical testbed (e.g., Raspberry Pi 4 nodes running OpenWRT with batman-adv or babeld) to account for real-world effects including RF interference, multi-path fading, asymmetric link quality, and CPU scheduling latency.

Security analysis. WMN routing protocols are vulnerable to attacks including black-hole attacks (a malicious node advertises routes to attract traffic and then drops it), wormhole attacks, and Sybil attacks. Extending the evaluation to include adversarial node behaviour would add critical practical relevance, particularly for the IoT security domain.

6.4 Final Remarks

This project demonstrated that simulation is an effective and reproducible method for evaluating the trade-offs between routing protocols in Wireless Mesh Networks. The OMNeT++/INET platform provided a high-fidelity model of the IEEE 802.11 MAC and physical layers, yielding results that closely reflect the mechanisms observed in real-world 802.11 networks.

The central finding — that the dominant performance bottleneck shifts from route availability to channel capacity as network density increases — is not specific to AODV and DSDV. It is a general principle that applies to any comparison between reactive and proactive routing protocols in any wireless network that uses a shared-medium MAC protocol. This principle should guide protocol designers and network engineers alike: the best routing protocol is the one whose control overhead is best matched to the channel capacity available at the target deployment density.

Wireless Mesh Networks continue to play a growing role in smart city infrastructure, industrial automation, agricultural monitoring, and emergency communications. As deployments grow larger and energy constraints become more stringent, the quantitative understanding provided by studies such as this one becomes increasingly important for making informed protocol-selection decisions.

Bibliography

[1] C. E. Perkins and P. Bhagwat, "Highly dynamic destination-sequenced distance-vector routing (DSDV) for mobile computers," in *Proceedings of the ACM SIGCOMM Conference*, London, UK, Aug. 1994, pp. 234–244.

[2] C. Perkins, E. Belding-Royer, and S. Das, "Ad Hoc On-Demand Distance Vector (AODV) Routing," IETF, RFC 3561, Jul. 2003.

[3] I. F. Akyildiz, X. Wang, and W. Wang, "Wireless mesh networks: a survey," *Computer Networks*, vol. 47, no. 4, pp. 445–487, Mar. 2005.

[4] R. Bruno, M. Conti, and E. Gregori, "Mesh networks: commodity multihop ad hoc networks," *IEEE Communications Magazine*, vol. 43, no. 3, pp. 123–131, Mar. 2005.

[5] M. Abolhasan, T. Wysocki, and E. Dutkiewicz, "A review of routing protocols for mobile ad hoc networks," *Ad Hoc Networks*, vol. 2, no. 1, pp. 1–22, Jan. 2004.

[6] T. Clausen and P. Jacquet, "Optimized Link State Routing Protocol (OLSR)," IETF, RFC 3626, Oct. 2003.

[7] D. B. Johnson, D. A. Maltz, and Y.-C. Hu, "The Dynamic Source Routing Protocol (DSR) for Mobile Ad Hoc Networks for IPv4," IETF, RFC 4728, Feb. 2007.

[8] G. Bianchi, "Performance analysis of the IEEE 802.11 distributed coordination function," *IEEE Journal on Selected Areas in Communications*, vol. 18, no. 3, pp. 535–547, Mar. 2000.

[9] S. R. Das, C. E. Perkins, and E. M. Royer, "Performance comparison of two on-demand routing protocols for ad hoc networks," in *Proceedings of IEEE INFOCOM*, Tel Aviv, Israel, 2000, pp. 3–12.

[10] D. Shukla and L. Mahajan, "Performance analysis of AODV, DSR, and DSDV routing protocols in mobile ad hoc networks with varying network load and speed," *International Journal of Computer Applications*, vol. 72, no. 17, pp. 11–16, 2013.

[11] A. Boukerche, "Performance evaluation of routing protocols for ad hoc wireless networks," *Mobile Networks and Applications*, vol. 9, no. 4, pp. 333–342, Aug. 2004.

[12] P. Chatzimisios, A. C. Boucouvalas, and V. Vitsas, "IEEE 802.11 packet delay — a finite retry limit analysis," in *Proceedings of IEEE GLOBECOM*, vol. 2, San Francisco, CA, 2003, pp. 950–954.

[13] R. Meszaros *et al.*, "INET Framework for OMNeT++, version 4.5," https://inet.omnetpp.org, 2023.

[14] A. Varga, "OMNeT++," in *Modeling and Tools for Network Simulation*, K. Wehrle, M. Güneş, and J. Gross, Eds. Berlin: Springer, 2010, pp. 35–59.

Appendix A

Simulation Configuration Files

A.1 Complete `omnetpp.ini`

The complete `omnetpp.ini` configuration file used in all simulations is listed below. Comments explain the purpose of each parameter group.

Listing A.1: Complete `omnetpp.ini` simulation configuration.

```
[General]
network = inet.examples.thesis_wmn.MeshNetwork
sim-time-limit = 50s
record-eventlog = false

# 1. DISABLE STATIC ROUTES (force AODV/DSDV to handle routing)
*.configurator.addStaticRoutes = false

# 2. NODE COUNT SWEEP
*.numNodes = ${N=10,20,30,40,50}

# 3. MOBILITY: Stationary (MassMobility with speed=0)
# Using MassMobility instead of StationaryMobility to avoid
# a macOS Qtenv crash in OMNeT++ 6.3 on Apple Silicon.
**.mobility.bounds = "1000m 1000m 0m"
**.mobility.typename        = "MassMobility"
**.mobility.initialX        = uniform(0m, 1000m)
**.mobility.initialY        = uniform(0m, 1000m)
**.mobility.initialZ        = 0m
**.mobility.speed           = 0mps
**.mobility.changeInterval  = 1s
**.mobility.angleDelta      = 0deg
```

```
**.mobility.initFromDisplayString = false

# Show radio range circles in the Qtenv GUI
**.wlan[*].radio.displayCommunicationRange = true

# 4. IEEE 802.11g RADIO MEDIUM
**.radioMedium.typename = "Ieee80211ScalarRadioMedium"
**.radioMedium.backgroundNoise.power = -90dBm
**.wlan[*].typename       = "Ieee80211Interface"
**.wlan[*].radio.typename = "Ieee80211ScalarRadio"
**.wlan[*].mac.typename   = "Ieee80211Mac"
**.wlan[*].mac.opMode     = "g(mixed)"
**.wlan[*].mac.bitrate    = 54Mbps
**.wlan[*].radio.transmitter.communicationRange = 100m

# 5. ENERGY MODEL
**.energyStorage.typename                                = "
   IdealEpEnergyStorage"
**.energyStorage.initialCapacity                         = 100J
**.wlan[*].radio.energyConsumer.typename                 = "
   StateBasedEpEnergyConsumer"
**.wlan[*].radio.energyConsumer.transmitterPowerConsumption =
   1.65W
**.wlan[*].radio.energyConsumer.receiverPowerConsumption    =
   1.4W
**.wlan[*].radio.energyConsumer.sleepPowerConsumption       =
   0.1W

# 6. TRAFFIC: Many-to-one UDP (all nodes -> host[0])
**.host[*].numApps = 1

# All nodes send to the sink
**.host[*].app[0].typename       = "UdpBasicApp"
**.host[*].app[0].destAddresses  = "host[0]"
**.host[*].app[0].destPort       = 5000
**.host[*].app[0].messageLength  = 1000B
**.host[*].app[0].sendInterval   = uniform(0.5s, 1s)
**.host[*].app[0].startTime      = uniform(1s, 5s)
**.host[*].app[0].stopTime       = 49s

# Sink receives on port 5000
```

```
**.host[0].app[0].typename   = "UdpSink"
**.host[0].app[0].localPort  = 5000

# Enable end-to-end delay recording at the sink
**.host*.app[0].endToEndDelay.result-recording-modes = +mean
**.host*.app[0].endToEndDelay.scalar-recording = true

# --- PROTOCOL CONFIGURATIONS ---

[Config AODV_Test]
description = "Simulate Reactive AODV (RFC 3561)"
**.routingApp.typename = "Aodv"

[Config DSDV_Test]
description = "Simulate Proactive DSDV (Perkins & Bhagwat 1994)"
**.routing.typename = "Dsdv"
```

A.2 Complete `MeshNetwork.ned`

Listing A.2: `MeshNetwork.ned` — network definition file.

```
package inet.examples.thesis_wmn;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
import inet.node.inet.ManetRouter;
import inet.physicallayer.wireless.common.medium.RadioMedium;

network MeshNetwork
{
    parameters:
        int numNodes;
        @display("bgb=1000,1000"); // 1000m x 1000m deployment area

    submodules:
        configurator: Ipv4NetworkConfigurator {
            @display("p=100,100");
        }
        radioMedium: RadioMedium {
```

```
18              @display("p=100,200");
19          }
20          host[numNodes]: ManetRouter {
21              @display("p=500,500");
22          }
23  }
```

Appendix B

Mathematical Background

B.1 Bianchi's IEEE 802.11 Saturation Throughput Model

Bianchi [8] derived a closed-form expression for the saturation throughput of IEEE 802.11 DCF. The model assumes:

- n nodes, each always having a packet to transmit (saturated conditions).
- Transmission probability τ per slot, derived from the steady-state analysis of the back-off Markov chain.
- No hidden terminals; all nodes within mutual range.

The key equations are:

$$\tau = \frac{2(1-2p)}{(1-2p)(W+1) + pW(1-(2p)^m)} \tag{B.1}$$

where p is the conditional collision probability, $W = \mathrm{CW}_{\min}$, and m is the maximum back-off stage such that $2^m W = \mathrm{CW}_{\max}$.

The collision probability is:

$$p = 1 - (1-\tau)^{n-1} \tag{B.2}$$

These equations form a system that can be solved iteratively for τ and p. The saturation throughput S is then:

$$S = \frac{P_s \cdot P_{tr} \cdot E[L]}{(1 - P_{tr})\sigma + P_{tr} P_s T_s + P_{tr}(1 - P_s) T_c} \tag{B.3}$$

where:

$$P_{tr} = 1 - (1 - \tau)^n \qquad \text{(probability at least one node transmits)} \tag{B.4}$$

$$P_s = \frac{n\tau(1 - \tau)^{n-1}}{P_{tr}} \qquad \text{(prob. of successful transmission)} \tag{B.5}$$

$$T_s = \text{DIFS} + T_{\text{frame}} + \text{SIFS} + T_{\text{ACK}} \qquad \text{(successful slot time)} \tag{B.6}$$

$$T_c = \text{DIFS} + T_{\text{frame}} \qquad \text{(collision slot time)} \tag{B.7}$$

For 802.11g at 54 Mbps with 1000 B payload, $T_{\text{frame}} \approx 200\,\mu$s. As n increases from 10 to 50, P_s decreases and T_c events become more frequent, causing S to decrease from near-maximum to well below 50% of the theoretical channel capacity. This is the mechanism that explains why DSDV's additional control-frame traffic causes disproportionate throughput degradation at $N = 50$.

B.2 Retry-Limit Frame Drop Probability

When the channel is saturated and the collision probability p is high, the probability that a frame survives all m_r retry attempts without an abort is:

$$P_{\text{delivery}} = (1 - p)^{m_r + 1} = (1 - p)^8 \qquad (m_r = 7) \tag{B.8}$$

Table B.1 shows how frame-drop probability varies with collision probability:

Table B.1: Frame-drop probability vs. collision probability ($m_r = 7$ retries).

Collision probability p	**Delivery probability**	**Drop rate**
0.01	0.923	7.7%
0.05	0.663	33.7%
0.10	0.430	57.0%
0.20	0.168	83.2%
0.30	0.058	94.2%
0.50	0.004	99.6%

This table demonstrates that even a modest increase in collision probability — from 5% to 10% — nearly doubles the drop rate. DSDV's routing broadcasts, by increasing the effective number of competing nodes, can push the network from the 5% to the 10–20% collision-probability regime, causing the catastrophic drop rates observed at $N = 50$.